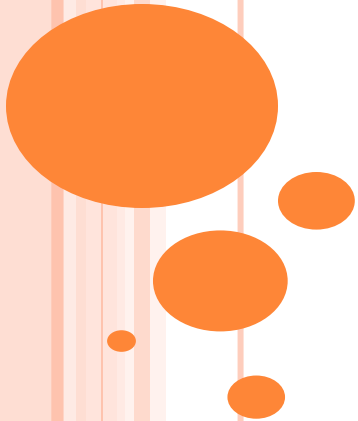
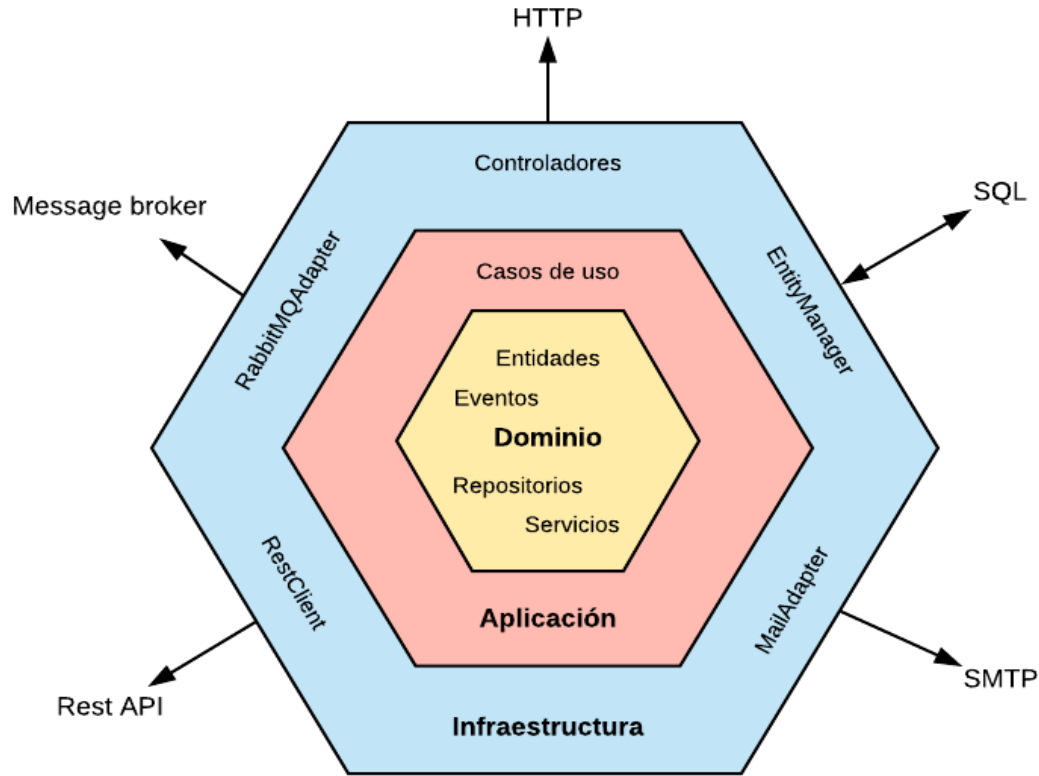


# OTROS PATRONES ACTUALES

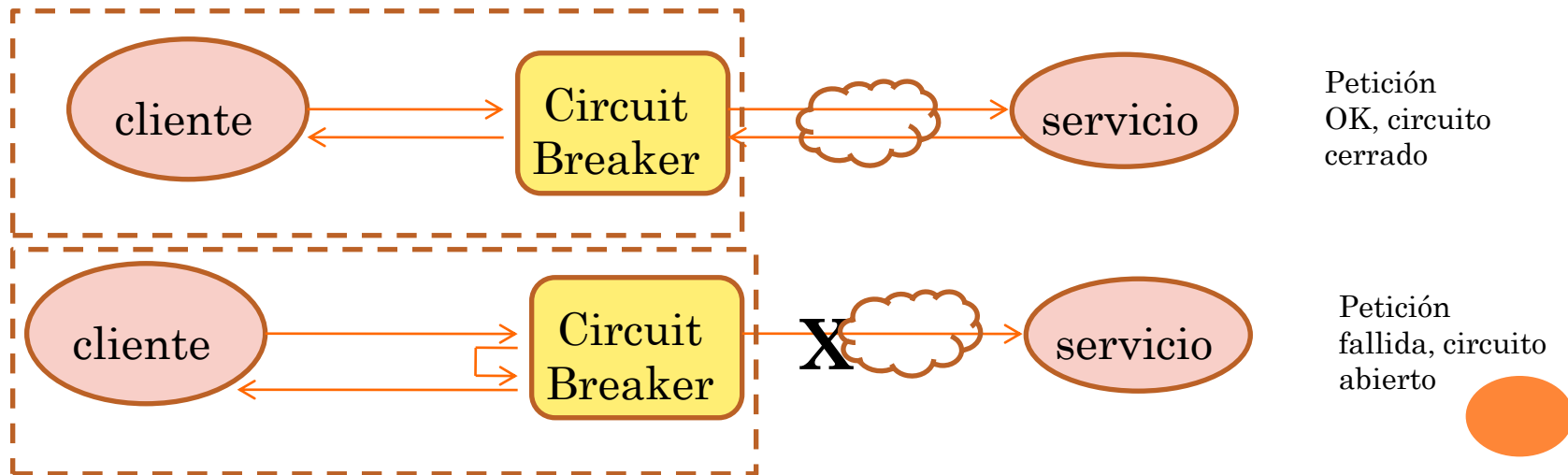


# Patrón Arquitectura Hexagonal



# Patrón Circuit Breaker

- Utilizado en contextos de recuperación ante fallos
- En interacción con microservicios, permite que el cliente pueda obtener respuesta ante fallos del servicio llamado



# Patrón SAGA

- Utilizado en sistemas distribuidos (especialmente en microservicios) para mantener la consistencia de los datos entre varios servicios sin usar transacciones distribuidas
  - En lugar de deshacer transacciones, se basa en aplicar operaciones de compensación.
  - De hecho, una saga es una secuencia de transacciones locales que pertenecen a diferentes servicios. Cada transacción:
    - Se ejecuta de forma independiente y atómica dentro de su propio servicio.
    - Si alguna de las transacciones falla, se ejecutan acciones compensatorias (operaciones inversas) para deshacer los efectos de las anteriores
- 

# Patrón CQRS

- El patrón CQRS (Command Query Responsibility Segregation) se basa en separar la responsabilidad de lectura (Queries) y la de escritura (Commands) dentro de una aplicación
- Con CQRS, dividimos responsabilidades: En lugar de tener un mismo modelo de dominio (y las mismas clases/repositorios) para leer y escribir datos, usamos modelos distintos (distintas clases e interfaces)

Command side            Crear, modificar, eliminar

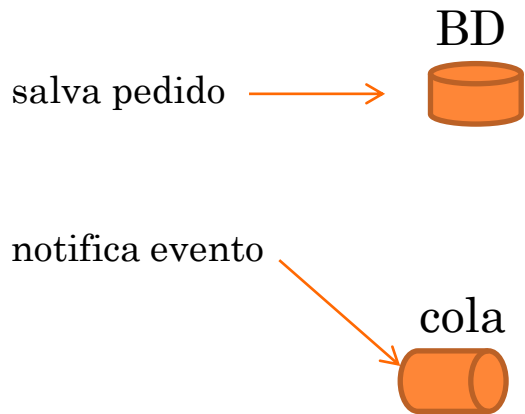
Query side            Consultar, buscar



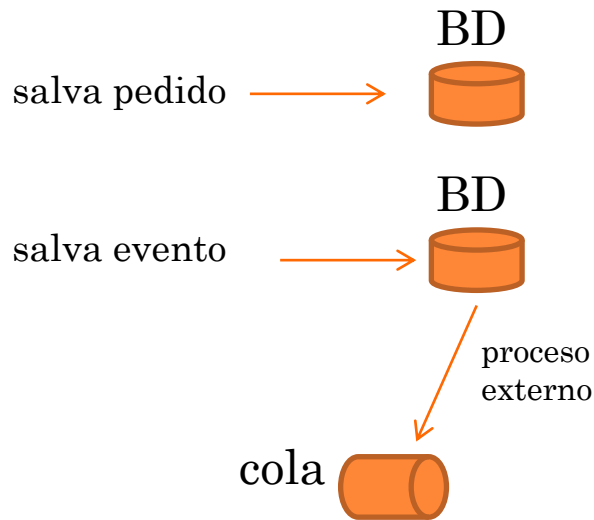
# Patrón Outbox

- Utilizando en contextos microservicios-mensajería
- Garantiza que la operación contra la base de datos y la de mensajería ocurren juntas

## *Tradicional*



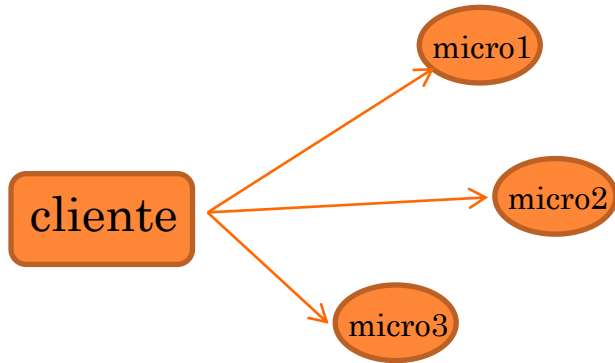
## *Patrón outbox*



# Patrón API Gateway

- Centraliza el acceso a varios microservicios desde un único punto
- El cliente no necesita conocer ubicación de los microservicios

*Sin gateway*



*Con gateway*

